EIE4428 Multimedia Communications (2026/27)

Mini-Project Report: Remote Video Monitoring

(Cover Sheet)

Group Member	Student ID
Julian Hugentobler	_____
_____	_____

Summary of the Project

1. **Development Platform:** Own implementation, **no WebRTC** and none of the other listed tools. Video server in **Python 3.12** (OpenCV 5 for capture/vision, FFmpeg libx264 + libopus through PyAV for compression, FastAPI/uvicorn for HTTP + WebSocket); web client in **React 19 + TypeScript** using the browser **WebCodecs API** (hardware H.264 decoding) over **WebSocket**; additional native viewer (Python). Developed and tested on Windows 11, Chrome / Edge.
2. **Functionalities of the System:**

Functions	Descriptions
Basic Remote Monitoring Function	Audio and video (live, two-way control)
Achieved Video Frame Rate (Frames/s)	30 frames/s at 1280×720 (configurable 5–60 fps)
Video Format	H.264 / AVC (Main profile), MJPEG fallback; recordings MP4
Video Resolution	640×360, 640×480, **1280×720** (default), 1920×1080
Audio Format (if any)	Opus, 48 kHz mono, 32 kbit/s
Concurrent Access to the Server	Yes, unlimited (encode-once fan-out; tested with 3 browsers + MJPEG)
Assessment via Internet	Yes (HTTPS + password login; port forwarding or Cloudflare Tunnel)

3. **Special Features of the System:**
   - **Ultra-low latency** live view: about 40–120 ms from capture to screen (measured with NTP-style clock sync and shown live).
   - **Motion detection** with alarm (red flashing frame, sound, desktop notification, event + snap-shot), adjustable sensitivity, illumination-change rejection.
   - **Object tracking** of moving objects with IDs and trajectories.
   - **Face detection, face recognition** (enroll people from the UI) and **emotion recognition** (7 expressions).
   - **Video recording with timing information**: manual or motion-triggered, with pre-roll/post-roll, millisecond OSD time stamp, in-browser playback and download.
   - **Audio** streaming with VU meter and **loud-noise alarm**.
   - **Event log** (SQLite) with filters and snapshots; **snapshots** in PNG/JPEG.
   - **Beautiful, responsive GUI**: digital PTZ (zoom 1–6×, pan), fullscreen, overlay toggles, live statistics with sparklines, live settings for resolution/fps/bit rate.
   - **Unlimited concurrent viewers** with per-viewer congestion control; **password login**, **HTTPS**; automatic **MJPEG fallback**; **native reception software**; standalone **.exe**.

Details of the Project

1. Achievements of the Project

We built a complete remote video monitoring system **from the ground up**. Our own code covers the whole multimedia chain: capture, compression, packetisation, transport, decoding and rendering. We used no streaming framework: no WebRTC, no media server, no player library. A user opens a normal web page on any PC (LAN or Internet) and sees the camera live in real time with sound, together with the results of the server-side video analytics.

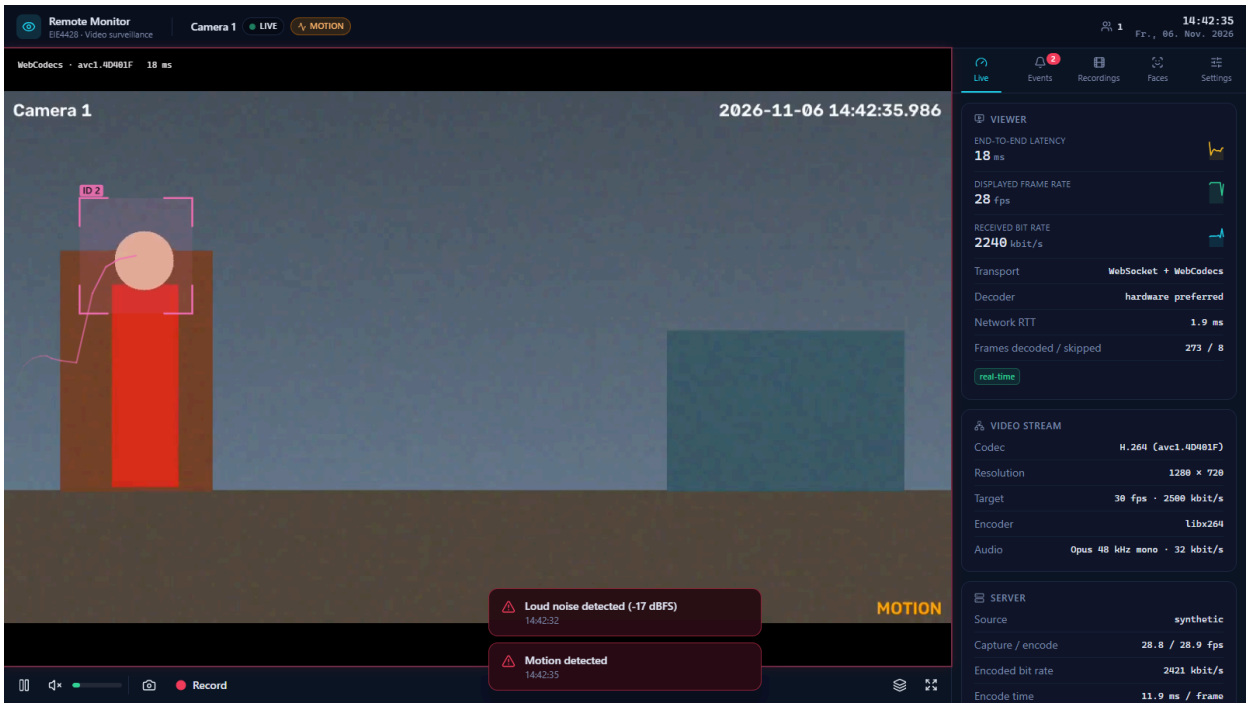


Figure 1: Web interface during a motion alarm: tracked object (ID, trajectory), OSD time stamp, latency chip (44 ms), live statistics and alarm notifications.

The major contributions of our group are:

1. **Own low-latency streaming protocol.** H.264 access units and Opus packets are framed with a 12-byte header (type, keyframe flag, 64-bit capture time stamp) and sent over one WebSocket per viewer. In the browser they are decoded by the hardware decoder through WebCodecs and painted onto a canvas. With this design we measured **30 fps** at **1280×720** with an **end-to-end latency of about 40 ms** on the same machine (≈ 120 ms worst case in our tests), **0 skipped frames** and a time-to-first-frame of ≈ 1.3 s including page load.
2. **Scalable server architecture.** Each frame is encoded once and the same bytes are sent to every viewer. Each viewer has its own queue with a congestion-control policy (drop to the next keyframe), so one slow client can never slow down the others. Concurrent access is unlimited and was tested with three independent browsers plus an MJPEG client.
3. **Intelligent monitoring.** Motion detection (Gaussian-mixture background subtraction) with an alarm state machine, multi-object tracking (alpha-beta filter), CNN face detection (YuNet), face recognition (SFace embeddings, enrollment from the UI) and facial-emotion recognition, all running live on the CPU in a separate thread.
4. **Complete product features.** Recording with pre-roll and timing information, event log, snapshots, audio with noise alarm, digital PTZ, password login, HTTPS, MJPEG fallback, a native viewer and a one-click executable.

5. **Quality assurance.** 17 automated server tests, unit tests for the web client and an automated browser test that drives a real (headless) Chrome and checks decoding, frame rate, latency, recording, concurrent viewers and the fallback mode.

2. Methodology

2.1 Development platform

Part	Technology (version) and reason for the choice
Server language	Python 3.12, managed with **uv**. Strong multimedia/vision ecosystem; heavy work runs in C libraries that release the GIL, so threads run truly in parallel.
Capture	OpenCV 5.0 `VideoCapture` (DirectShow, MJPG FourCC for HD at full frame rate).
Compression	FFmpeg 7 via PyAV 19: **libx264** (H.264) and **libopus** (Opus). We configure the codecs ourselves (zero-latency tuning, GOP, VBV, forced IDR); FFmpeg is used only as a codec library.
Computer vision	OpenCV MOG2 background subtraction; OpenCV DNN with YuNet, SFace and MobileFaceNet-FER models (OpenCV Model Zoo).
Web server	FastAPI + uvicorn (ASGI): REST API, WebSocket endpoint, static file serving, automatic OpenAPI docs.
Web client	React 19, TypeScript 7, Vite 8, Tailwind CSS 4, zustand; browser APIs **WebCodecs** (VideoDecoder/AudioDecoder), WebSocket, Canvas 2D, Web Audio, Notifications.
Tools	Git, ruff (lint), pytest, vitest, Playwright (browser test), PyInstaller (executable), Typst (this report).

Why not WebRTC? Besides the restriction in the project sheet, WebRTC would hide exactly the parts we wanted to learn and control: codec parameters, packetisation, time stamps and congestion handling. We compared HLS/DASH (2–6 s latency, too slow), fMP4 + MSE (extra buffering) and MJPEG (5–10× the bandwidth of H.264). We chose **H.264 Annex-B over WebSocket into WebCodecs** because it gives frame-level control and the lowest latency. MJPEG is kept as a fallback for browsers without WebCodecs.

2.2 Block diagram

Figure 2 shows the system. **Green blocks are our own code**; grey blocks are hardware or third-party libraries that we use and configure.

Data flow. The capture thread puts the newest frame into a one-element mailbox (`FrameSlot`). The encoder thread and the analytics thread both take only the **latest** frame. A slow stage therefore drops frames instead of building up delay, and the analytics can never lower the streamed frame rate. Encoded packets, audio packets and analytics results are handed to the asyncio event loop, where the **Hub** distributes them to the viewers. All time stamps are wall-clock microseconds at capture time, which lets every viewer measure the true end-to-end latency.

3. Implementation Details of the Project

All source code listed below was written by our group (≈ 3 200 lines of Python and ≈ 2 900 lines of TypeScript/TSX and tests). Libraries are only used through their public APIs.

3.1 Source files and their purpose

File (server/src/remote_monitor)	Purpose
`capture.py`	Camera/file/synthetic sources; capture thread with reconnect and back-off; `FrameSlot` latest-frame mailbox; frame-rate limiter.

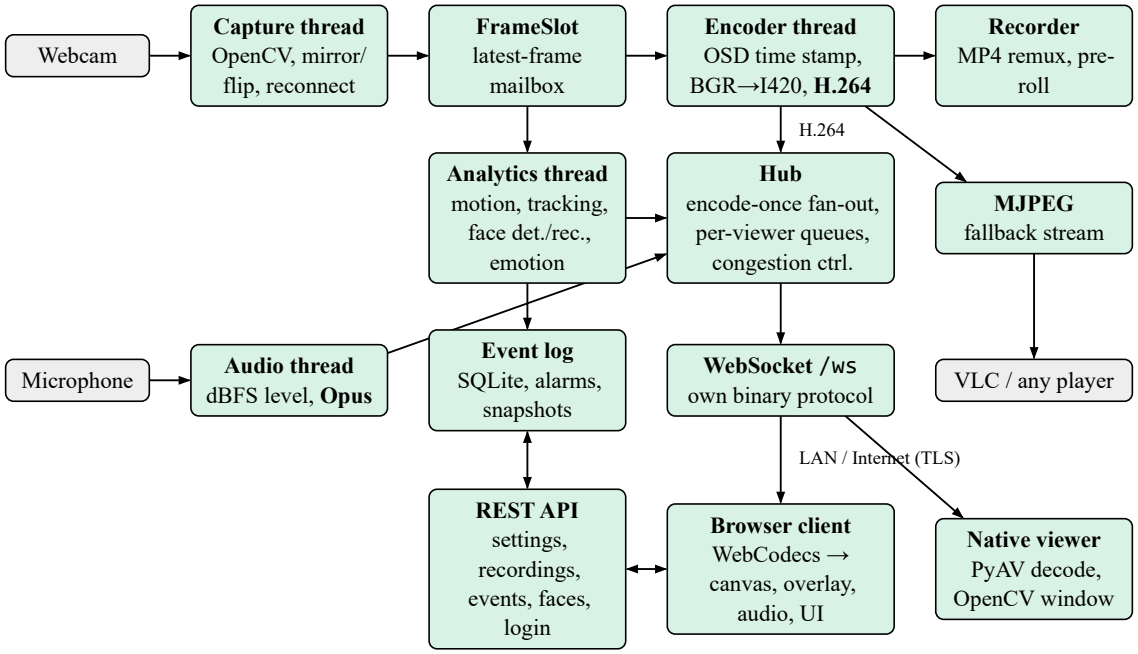


Figure 2: Block diagram of the system. Green = own work, grey = hardware / third-party.

`encoder.py`	H.264 encoder (libx264 `veryfast` + `zerolatency`, Main profile, VBV, forced IDR, hardware-encoder fallback); Annex-B NAL parser; RFC 6381 codec string from the SPS.
`audio.py`	PortAudio capture in 20 ms blocks, dBFS level, Opus encoder (`lowdelay`).
`protocol.py`, `hub.py`	12-byte binary media header; fan-out hub with per-viewer send queues, drop-to-keyframe congestion control, MJPEG subscribers.
`analytics/motion.py`	MOG2 background subtraction, morphology, contour boxes, alarm hysteresis, illumination-change rejection, box merging.
`analytics/tracker.py`	Multi-object tracker: greedy assignment (distance − IoU), alpha-beta filter, confirmation/deletion, trajectories.
`analytics/faces.py`	YuNet detection, 5-point alignment, SFace embedding + cosine matching against the enrolled gallery, 7-class emotion classifier, model download.
`osd.py`, `recorder.py`	Millisecond time-stamp OSD; MP4 recorder (remux, no re-encoding) with a two-GOP pre-roll ring buffer, post-roll, splitting.
`service.py`	Wires all threads together; alarm/event logic; statistics; live settings.
`app.py`, `auth.py`, `certs.py`	REST + WebSocket + MJPEG endpoints; HMAC session tokens; self-signed certificate with all LAN IPs.
`events.py`, `config.py`, `cli.py`, `viewer.py`	SQLite event log; validated, persisted settings; command line; native viewer.
web/src/lib	`videoPlayer.ts` (WebCodecs H.264 player with recovery logic), `audioPlayer.ts` (Opus + jitter buffer), `clock.ts` (NTP-style sync), `connection.ts` (WebSocket, reconnect, routing), `overlay.ts` (detections).
web/src/components, panels	Video stage with digital PTZ, control bar, live statistics, events, recordings, faces, settings, login, notifications.

3.2 Capture and compression

Frames come from the webcam through DirectShow with the MJPG FourCC; USB webcams deliver 720p/1080p at full frame rate only in MJPG. The encoder thread draws the OSD (camera name, date and time with milliseconds, REC/MOTION flags), converts BGR to YUV 4:2:0 with OpenCV (SIMD) and encodes with libx264:

```
ctx.options = {
    "preset": "veryfast", "tune": "zerolatency", "profile": "main", "forced-idr": "1",
```

```
    "x264-params": f"scenecut=0:vbv-maxrate={kbps}:vbv-bufsize={kbps // 2}:repeat-headers=1",
}
```

`zerolatency` disables B-frames and look-ahead, so **every frame produces exactly one packet immediately** and the encoder adds no delay. The VBV buffer (0.5 s) caps bit-rate spikes. The GOP is 2 s, which gives good compression. In addition, the server **forces an IDR frame on demand**: when a viewer joins, recovers, or a recording starts (rate-limited to one per 0.3 s). SPS/PPS are repeated in-band, and the server parses the SPS to derive the exact codec string (for example `avc1.4D401F` = Main profile, level 3.1). The browser needs this string to configure its decoder. Audio is captured in 20 ms blocks and compressed with Opus at 32 kbit/s.

3.3 Transport and congestion control

Each media message is `[type | flags | reserved | 64-bit time stamp | payload]`. JSON messages carry the decoder configuration, analytics metadata (≈ 15 Hz), statistics (1 Hz), events and clock-sync pongs. Because TCP never drops data, a slow viewer would accumulate an ever-growing delay. Our hub therefore monitors each viewer's queue. If more than ≈ 1 s of video is waiting, the queued video is discarded, a keyframe is requested and the viewer resumes at that IDR frame. This bounds latency and never corrupts the picture, because a P-frame is never decoded without its reference. Control messages are never dropped.

3.4 Browser decoding and rendering

`videoPlayer.ts` configures a `VideoDecoder` with `optimizeForLatency: true` and prefers hardware acceleration. It decodes each access unit as soon as it arrives (no jitter buffer) and paints the output `VideoFrame` onto a canvas. Robustness measures:

- decoding starts only at a keyframe;
- an IDR frame that arrives during configuration is stashed and used;
- the decoder is rebuilt after an error or when the browser reclaims it;
- the player skips to the next keyframe when the decode queue backs up;
- decoding pauses while the tab is hidden;
- keyframe requests are throttled to one per second.

Opus packets are decoded with `AudioDecoder` and scheduled on the Web Audio timeline with a 60 ms jitter buffer, re-anchored when the delay exceeds 300 ms. **Latency measurement**: the client sends its time c, the server answers with its time s, and the offset is $s - \left(c + \frac{\mathrm{RTT}}{2}\right)$ (minimum-RTT sample of the last 8). For each painted frame, latency = client time + offset − capture time stamp.

3.5 Analytics, events and recording

Motion: MOG2 runs on a 320-pixel grey image. Morphological opening and dilation clean the mask, contours give boxes, and a minimum area derived from the sensitivity slider filters small blobs. An alarm with hold-time hysteresis avoids flickering alarms. Frames with more than 60 % foreground are recognised as illumination changes, and the background model is relearned instead of raising an alarm. **Tracking:** merged boxes are matched to tracks using distance − IoU, with alpha-beta prediction; a track is confirmed after 3 hits and removed after 8 misses. **Faces:** YuNet detects faces with 5 landmarks. The aligned 112×112 crop feeds both SFace (128-D embedding, cosine threshold 0.363) and the emotion network. In our check, an enrolled face was still recognised after flipping and rescaling (similarity 0.86), while another person scored 0.13. All results are sent with normalised coordinates and drawn as a vector overlay, so each viewer can switch the overlays on or off.

Alarm events (motion, loud noise, unknown person) are stored in SQLite with a snapshot that has the detection boxes drawn in. They are pushed to every viewer as a toast, a beep and an optional desktop notification. **Recording** remuxes the already-encoded H.264 + Opus packets into MP4 without re-

encoding. A ring buffer holds the last two GOPs, so a motion-triggered recording starts **before** the motion and always begins with an IDR frame.

3.6 Evaluation

Measurement (Intel laptop, 12 threads, Windows 11, Chrome)	**Result**
Capture / encode frame rate, 1280×720	28.6–30 fps with all analytics enabled; webcam: 15 fps (camera limit in low light)
H.264 encode time per 720p frame	9–19 ms (libx264, CPU)
Bit rate at the 2 500 kbit/s setting	2.3–2.5 Mbit/s measured at the receiver
Displayed frame rate in browser	30 fps, 0 frames skipped
End-to-end latency (capture → canvas)	≈ 40 ms typical, ≤ 120 ms observed
Time to first frame (incl. page load)	≈ 1.3 s
Analytics rate / cost	13–15 Hz, ≈ 37 ms per run (motion + tracking + 3 CNNs)
Concurrent viewers	3 browsers + 1 MJPEG client simultaneously, unaffected
Automated tests	17 server tests, 5 web tests, browser end-to-end test: all passed
Supported formats	H.264 (Main), MJPEG, Opus, MP4 recordings (H.264 + Opus), JPEG/PNG snapshots

The settings panel exposes the main trade-off of video compression: a higher bit rate or a shorter GOP gives better quality and faster recovery, but costs more bandwidth.

4. Design of User Interface

The interface (Figure 3) follows the layout of commercial surveillance software, with a large live view and a control column. It uses a dark “security console” theme so the video stands out and the screen does not dazzle in a dark control room.

- **Header:** camera name, live status (LIVE / RECONNECTING), REC and MOTION indicators, number of viewers, clock.
- **Video stage:** live canvas with the analytics overlay (tracks with IDs and trajectories, motion regions, faces with name and emotion); digital PTZ pad (zoom 1–6× with buttons or mouse wheel, pan by dragging or with arrows); fullscreen by double-click; codec and latency chips; a red pulsing frame during alarms.
- **Control bar:** pause, listen/mute with volume and a live VU meter, snapshot (PNG including overlays), record button with running time, overlay toggles, fullscreen.
- **Side panel (tabs):**
  - **Live:** latency, frame rate and bit-rate sparklines; stream and server statistics.
  - **Events:** filterable timeline with snapshots that enlarge on click.
  - **Recordings:** thumbnails, in-browser playback, download, delete.
  - **Faces:** enroll people from the live camera.
  - **Settings:** resolution, frame rate, bit rate, GOP, mirror/flip, OSD, motion sensitivity, auto-record, face features, audio alarm, and per-viewer preferences.
- **Feedback:** toast notifications, an alarm beep and desktop notifications, an optional login page, and a responsive layout (on narrow screens the panel moves below the video).

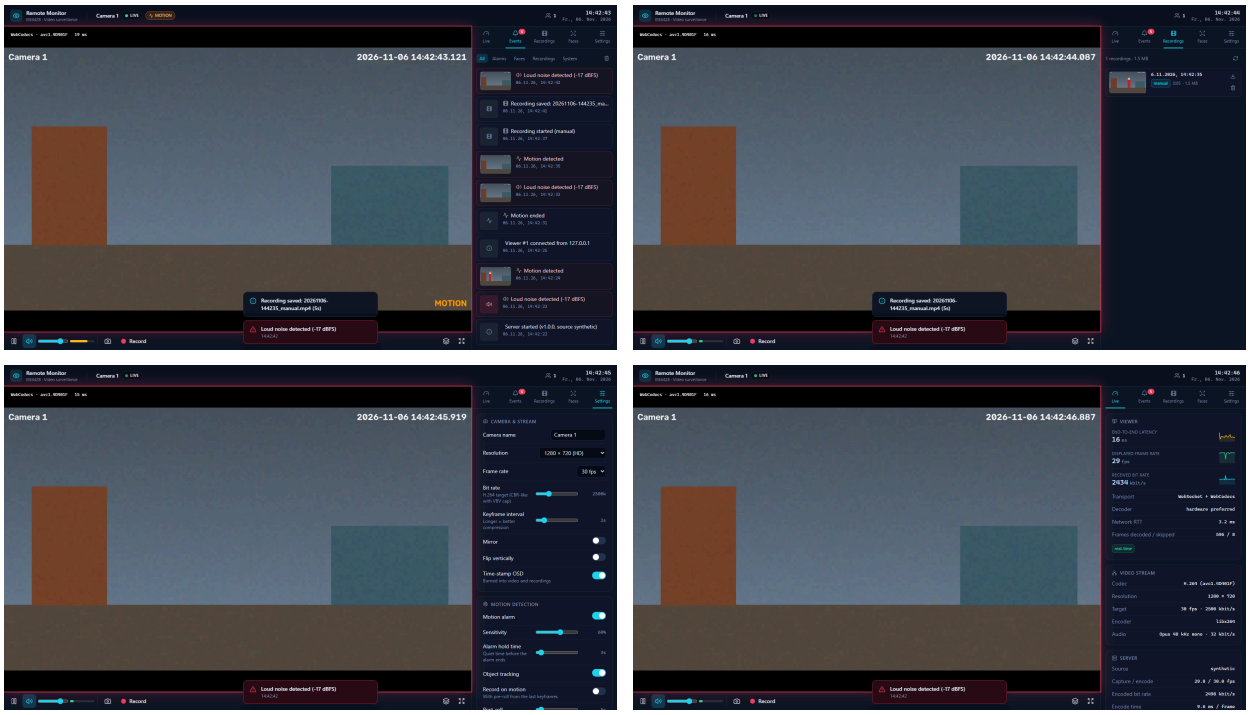


Figure 3: Event timeline with alarm snapshots, recordings library, live settings and live statistics.

5. Co-operation between Group Members

Member	Role and main contributions	Share
Julian Hugentobler	Server media pipeline: capture, H.264/Opus compression, binary protocol, hub and congestion control, recording, HTTPS/ authentication, native viewer, packaging; system architecture and documentation.	50 %
_____	Web client: WebCodecs video/audio players, clock sync and latency measurement, user-interface design and all panels; video analytics (motion detection, tracking, face and emotion recognition); testing and evaluation.	50 %
Both	Requirement analysis, protocol design, integration, demonstration preparation and this report.	–

We agreed on the WebSocket protocol (`protocol.py` / `protocol.ts`) first, so the two parts could be developed in parallel against the synthetic video source. We used Git for version control, and integration was verified with the automated end-to-end tests.

6. Conclusions

The project shows that a modern browser is enough as a high-performance reception client: with WebCodecs, our own WebSocket protocol delivers H.264 video at 30 fps with latency in the tens of milliseconds, without WebRTC or a media server. The most important lessons were these:

- in a live system, **latency must be bounded explicitly** (latest-frame hand-off, zero-latency encoding, drop-to-keyframe congestion control);
- **keyframe management** (GOP length versus on-demand IDR frames) decides both the bandwidth and how fast a viewer can join or recover;
- computer-vision workloads must be **decoupled** from the media path so they cannot lower the video frame rate.

Suggestions for further improvement:

1. **WebTransport** (QUIC datagrams) or FEC/NACK, to avoid TCP head-of-line blocking on lossy Wi-Fi or mobile links.
2. **Adaptive bit rate / simulcast** per viewer.
3. **Multi-camera** support with a grid view.
4. **Two-way audio** (talk-back).
5. A DNN **person detector** to reduce false alarms from moving curtains or trees.
6. Multi-user accounts with roles.
7. **Hardware encoding** by default (NVENC/QSV are already selectable) to reduce CPU load.

7. References

[1] W3C, "WebCodecs," W3C Working Draft. https://www.w3.org/TR/webcodecs/

[2] MDN Web Docs, "WebCodecs API" and "WebSocket API." https://developer.mozilla.org/en-US/docs/Web/API/WebCodecs_API

[3] ITU-T Rec. H.264 | ISO/IEC 14496-10, "Advanced video coding for generic audiovisual services."

[4] IETF RFC 6455, "The WebSocket Protocol," 2011. https://www.rfc-editor.org/rfc/rfc6455

[5] IETF RFC 6716, "Definition of the Opus Audio Codec," 2012. https://www.rfc-editor.org/rfc/rfc6716

[6] IETF RFC 6381, "The 'Codecs' and 'Profiles' Parameters for 'Bucket' Media Types," 2011.

[7] D. L. Mills, IETF RFC 5905, "Network Time Protocol Version 4," 2010 (clock-offset estimation).

[8] FFmpeg project and x264 documentation. https://ffmpeg.org/ · https://www.videolan.org/developers/x264.html

[9] PyAV - Pythonic bindings for FFmpeg. https://pyav.basswood-io.com/

[10] OpenCV 5 documentation, `BackgroundSubtractorMOG2`, `FaceDetectorYN`, `FaceRecognizerSF`. https://docs.opencv.org/

[11] Z. Zivkovic, "Improved adaptive Gaussian mixture model for background subtraction," ICPR, 2004.

[12] W. Wu, H. Peng, S. Yu, "YuNet: A Tiny Millisecond-level Face Detector," Machine Intelligence Research, 2023.

[13] Y. Zhong et al., "SFace: Sigmoid-Constrained Hypersphere Loss for Robust Face Recognition," IEEE TIP, 2021.

[14] OpenCV Model Zoo (YuNet, SFace, facial expression recognition). https://github.com/opencv/opencv_zoo

[15] FastAPI https://fastapi.tiangolo.com/ · uvicorn https://www.uvicorn.org/

[16] React https://react.dev/ · Vite https://vite.dev/ · Tailwind CSS https://tailwindcss.com/

[17] Cloudflare Tunnel documentation (Internet access). https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/

[18] EIE4428 Mini-Project sheet "Remote Video Monitoring," PolyU EEE, 2026.

[19] Project documentation: `docs/ARCHITECTURE.md` in the submitted source code.